Credit Card Fraud Detection Using SMOTE and Cost-Sensitive XGBoost

Sharmeen Fatima
Student of Software Engineering
Department of Computer Science / Machine Learning
It Simplera Solutions
Intern ID: AIMLB01-1994

***Abstract*—Financial fraud detection remains a critical challenge for modern electronic payment ecosystems due to the extreme class imbalance inherent in transactional data, where legitimate transactions overwhelmingly outnumber fraudulent ones. This study implements and compares four classifiers on the benchmark Kaggle Credit Card Fraud Detection dataset (284,807 transactions, 0.173% fraud): Logistic Regression, Random Forest, a standard XGBoost baseline, and a proposed SMOTE-enhanced, Cost-Sensitive XGBoost model. Using an 80/20 stratified split, the baseline models achieved F1-scores of 0.7241 (Logistic Regression), 0.8380 (Random Forest), and 0.8387 (XGBoost baseline), with ROC-AUC values of 0.9573, 0.9704, and 0.9191 respectively. The proposed model, trained on SMOTE-rebalanced data with a dynamic `scale_pos_weight` penalty, achieved the highest ROC-AUC (0.9764) and the highest fraud recall (89.8%), successfully catching more actual fraud cases than any baseline. However, at the default 0.5 decision threshold, the same model's precision on the fraud class collapsed to 5.1% (F1-score of 0.0969), producing a large number of false positives. This result is analyzed in detail as a central finding of the study: it empirically demonstrates why ROC-AUC alone can be a misleading metric on severely imbalanced data, and it motivates decision-threshold calibration as an essential, non-optional step when deploying SMOTE-trained models — a step implemented in the accompanying Streamlit deployment application via an operational probability threshold.**

***Index Terms*—Financial Fraud Detection, Class Imbalance, SMOTE, Cost-Sensitive XGBoost, Streamlit Deployment, Machine Learning.**

I. Introduction

The rapid proliferation of digital payment platforms, mobile wallets, and e-commerce services has positioned financial fraud as a multi-billion-dollar threat on a global scale. Detecting fraudulent transactions in real time presents a considerable computational and statistical challenge, given that fraudulent activity typically accounts for a tiny fraction of total transaction volume — in the dataset used in this study, only 0.173%. This severe class imbalance means that a naive classifier which labels every transaction as legitimate can still achieve accuracy in excess of 99.8%, while being practically useless for fraud interception.

Traditional rule-based detection systems are inherently limited in their ability to adapt to continuously evolving fraudulent behavioral patterns. Machine learning techniques offer greater adaptability by learning latent patterns directly from historical transaction data; however, models trained naively on severely imbalanced datasets tend to exhibit either low recall (missing fraud) or, as this study demonstrates, a collapse in precision once resampling is introduced without corresponding threshold calibration.

This paper addresses class imbalance through a combination of data-level oversampling (SMOTE), applied strictly within the training partition, and algorithmic cost sensitivity via XGBoost's `scale_pos_weight` parameter, and reports results *exactly as produced by the accompanying experimental notebook*, including a significant precision/recall trade-off issue observed in the final model at its default decision threshold.

A. Motivation

The motivation for this work stems from three practical observations:

1) Standard classification metrics such as accuracy are misleading on datasets with a fraud prevalence below 1%.
2) ROC-AUC, while widely reported, can remain high even when a model's precision at a practical operating threshold is very poor — a caution raised by Fawcett [3] and empirically confirmed by the results of this study.
3) Combining SMOTE with a cost-sensitive classifier changes the model's calibration, meaning the default 0.5 probability threshold used for baseline models is no longer appropriate and must be re-tuned for deployment.

B. Contributions

The principal contributions of this work are as follows:

1) A controlled, reproducible comparison of Logistic Regression, Random Forest, standard XGBoost, and SMOTE-enhanced Cost-Sensitive XGBoost on the same stratified train/test split.
2) Implementation of SMOTE exclusively on the training partition to avoid data leakage, combined with a dynamically computed cost-sensitivity weight w_{pos}.
3) An honest, quantitative analysis of the precision collapse observed in the proposed model at the default threshold, and its implications for real-world deployment.
4) An end-to-end deployment of the trained model as an interactive Streamlit application implementing an operational decision threshold (15%) to mitigate the precision issue identified during evaluation.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the dataset and preprocessing. Section IV presents the methodology. Section V details the experimental setup. Section VI reports the results exactly as produced by the notebook. Section VII discusses these results, including the precision/recall trade-off and other limitations. Section VIII describes the deployment architecture. Section IX concludes and outlines future work.

II. Related Work

Chawla et al. (2002) — SMOTE [1]. Introduced the Synthetic Minority Over-sampling Technique, generating synthetic minority-class samples via interpolation between existing minority instances and their k-nearest neighbors, rather than duplicating samples. SMOTE forms the data-level imbalance-handling component of the proposed pipeline in this study.

Chen and Guestrin (2016) — XGBoost [2]. Introduced XGBoost, a scalable, regularized implementation of gradient-boosted decision trees with native support for class-weighting via `scale_pos_weight` — directly used in this study's cost-sensitive strategy.

Fawcett (2006) — ROC Analysis [3]. Cautioned that ROC curves can present an overly optimistic picture of classifier performance on highly imbalanced datasets, since the false-positive rate is diluted by a large number of true negatives. This study's own results (Section VII) provide a concrete, empirical illustration of exactly this failure mode: the proposed model attains the highest ROC-AUC of all four classifiers (0.9764) while simultaneously having the worst F1-score (0.0969) due to a collapsed precision.

Wei et al. (2013) — Cost-Sensitive Deep Learning [4]. Investigated cost-sensitive learning for asymmetric credit card fraud data, showing that asymmetric cost matrices reduce financial losses from false negatives, generally at some expense to precision — consistent with the recall/precision trade-off observed in this study.

Carcillo et al. (2018) — Scarff [5]. Proposed a scalable streaming framework combining SMOTE with ensemble classifiers for near-real-time fraud detection, and reported a consistent trade-off between precision degradation and recall improvement as resampling ratios increase — the same trade-off observed here, but manifesting far more severely due to the additional `scale_pos_weight` penalty stacked on top of full SMOTE rebalancing.

A. Research Gap Addressed

Most existing literature focuses on offline algorithmic performance without addressing the decision-threshold calibration required once resampling changes a model's output distribution, nor does it always report the full precision/recall consequences of combining multiple imbalance-handling techniques (SMOTE *and* cost-sensitive weighting simultaneously). This project addresses that gap directly by reporting the unfiltered outcome of this combination, analyzing why it occurs, and demonstrating a practical mitigation (threshold tuning) in the deployed application. Table I summarizes the gap left open by each reviewed paper.

TABLE I: Summary of Reviewed Literature and Gaps Addressed

Paper	Contribution	Gap Left Open
Chawla et al. [1]	Introduced SMOTE oversampling	No cost-sensitive or deployment component
Chen & Guestrin [2]	Introduced XGBoost	Not evaluated on imbalanced fraud data specifically
Fawcett [3]	PR vs. ROC analysis	No proposed model or empirical demonstration on a real dataset
Wei et al. [4]	Cost-sensitive deep learning	No resampling, no threshold-calibration analysis
Carcillo et al. [5]	SMOTE + ensembles, streaming	No stacked cost-sensitive boosting, no reported precision collapse

III. Dataset Description & Preprocessing

This study uses the Kaggle Credit Card Fraud dataset[1], released by the Machine Learning Group at Université Libre de Bruxelles (ULB), comprising 284,807 European cardholder transactions. The dataset contains 30 numerical input features and a binary target label:

- **V1–V28**: Principal components derived via PCA, anonymizing sensitive transaction details.
- **Amount & Time**: The only two raw (non-PCA) attributes.
- **Class**: Target label — 0 = legitimate ($n = 284{,}315$), 1 = fraudulent ($n = 492$, 0.173% of the dataset).

A. Preprocessing Pipeline

1) **Standardization.** `Amount` and `Time` were each standardized independently using separate `StandardScaler` instances (`scaler_amount`, `scaler_time`), producing `scaled_amount` and `scaled_time`; the original `Amount` and `Time` columns were then dropped.
2) **Stratified train-test split.** An 80/20 stratified split (on `Class`) produced a training set of 227,845 transactions and a test set of 56,962 transactions, preserving the 0.173% fraud ratio in both partitions.
3) **SMOTE oversampling.** SMOTE was applied *exclusively* to the training partition (X_{train}), after the split, to prevent data leakage into the test set.

IV. Methodology & Architecture

A. Baseline Models

Three baseline classifiers were trained directly on the (non-resampled) standardized training data:

- **Logistic Regression**: `max_iter=1000`, default regularization.

[1] https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

- **Random Forest**: `n_estimators=50`, `max_depth=10`.
- **XGBoost (Baseline)**: `n_estimators=50`, `max_depth=6`, `eval_metric='logloss'` — trained *without* any class-imbalance handling, to isolate the effect of the imbalance-handling techniques applied in the proposed model.

All three baselines used `random_state=42`.

B. Proposed Model: SMOTE + Cost-Sensitive XGBoost

SMOTE (`random_state=42`) was applied to the training split, and a cost-sensitivity weight was computed dynamically as:

$$w_{pos} = \frac{N_{negative}^{train}}{N_{positive}^{train}} \tag{1}$$

where $N_{negative}^{train}$ and $N_{positive}^{train}$ are the legitimate and fraudulent transaction counts in the *original* (pre-SMOTE) training set. An XGBoost classifier was then trained on the SMOTE-resampled data with this weight passed to `scale_pos_weight`, using `n_estimators=200`, `max_depth=6`, `learning_rate=0.05`, `eval_metric='logloss'`, and `random_state=42`. The trained model and the amount scaler were serialized to `fraud_model.pkl` and `scaler.pkl` respectively for downstream deployment.

Because this model combines *two* imbalance-handling mechanisms simultaneously — full SMOTE rebalancing of the training set *and* an XGBoost-internal cost penalty tuned for the original (unbalanced) class ratio — its predicted probabilities are pushed strongly toward the fraud class relative to the baselines, with direct consequences for the default-threshold results reported in Section VI.

V. Experimental Setup

A. Environment & Reproducibility

Experiments were implemented in Python using `pandas`, `numpy`, `scikit-learn`, `xgboost`, `imbalanced-learn`, and `joblib`, executed in a Google Colab notebook environment. A global random seed (`RANDOM_STATE = 42`) was applied to the train-test split, the SMOTE sampler, and all model initializations to ensure reproducibility. Two artifacts were serialized at the end of the training pipeline: `fraud_model.pkl` (the fitted XGBoost model) and `scaler.pkl` (the fitted `StandardScaler` for the `Amount` feature), both loaded directly by the Streamlit deployment application described in Section VIII, ensuring that the exact model evaluated in Section VI is the one served in production.

B. Validation Strategy

An 80/20 stratified train-test split was used (227,845 / 56,962 transactions). Per the experimental design documented in the notebook, 5-Fold Stratified K-Fold cross-validation was intended for model selection; the results reported in Section VI reflect performance on the single held-out stratified test set, as computed in the model evaluation cells.

C. Evaluation Metrics

Models were compared primarily using **F1-Score** and **ROC-AUC**, computed via `f1_score` and `roc_auc_score` on the held-out test set. For the proposed model, a full `classification_report` (precision, recall, F1, and support per class) was additionally generated. Precision-Recall/AUPRC analysis — flagged in the experimental design as an intended metric, consistent with Fawcett's [3] recommendation for imbalanced data — was not computed as a numeric summary in the final comparison table of the current notebook version; this is noted explicitly as a limitation in Section VII and as a direction for immediate future work.

VI. Results & Evaluation

A. Baseline Diagnostic Visualizations

Prior to training the proposed model, the notebook generates two diagnostic visualizations for the three baseline classifiers: (1) an ROC-curve comparison plot (Logistic Regression, Random Forest, XGBoost Baseline, overlaid with a random-chance diagonal), and (2) a side-by-side panel of confusion matrices for the same three models, using `seaborn.heatmap` on the raw `sklearn.metrics.confusion_matrix` output. These plots are produced directly from `roc_curve()` and `confusion_matrix()` calls in the notebook (Milestone 2, Part 6) and are intended to visually confirm that, prior to any imbalance handling, all three baselines separate the classes reasonably well in ranking terms (ROC-AUC between 0.919 and 0.970) while their exact false-positive/false-negative trade-offs differ with model complexity. The underlying per-cell confusion-matrix counts were not printed as text output in the executed notebook cells, so only the aggregate F1-Score and ROC-AUC values (Table II) are reproduced numerically in this paper; the plots themselves remain available in the notebook for visual inspection.

B. Quantitative Comparison

Table II reports the F1-Score and ROC-AUC for all four models exactly as produced by the evaluation cell of the notebook.

TABLE II: Model Performance Comparison on Held-Out Test Set (56,962 transactions)

Model	F1-Score	ROC-AUC
Logistic Regression	0.7241	0.9573
Random Forest	0.8380	0.9704
XGBoost Baseline (no imbalance handling)	0.8387	0.9191
SMOTE + Cost-XGB (Proposed)	**0.0969**	**0.9764**

Fig. 1 visualizes both metrics side by side. The proposed model attains the highest ROC-AUC of all four classifiers, yet its F1-score is an order of magnitude lower than every baseline — the central finding of this study, examined in detail in Section VII.

Fig. 1: F1-Score and ROC-AUC comparison across all four evaluated models, using the exact values produced by the experimental notebook.

C. Detailed Classification Report — Proposed Model

Table III reproduces the full classification report for the proposed SMOTE + Cost-Sensitive XGBoost model on the test set, at the default 0.5 probability threshold used by `.predict()`.

TABLE III: Classification Report — SMOTE + Cost-XGB (Test Set, threshold = 0.5)

Class	Precision	Recall	F1	Support
0 (Legitimate)	1.00	0.97	0.99	56,864
1 (Fraud)	0.05	0.90	0.10	98
Accuracy		0.97		56,962
Macro avg	0.53	0.93	0.54	56,962
Weighted avg	1.00	0.97	0.98	56,962

The model correctly recovers 90% of fraudulent transactions (recall), consistent with the intent of SMOTE and cost-sensitive weighting, but only 5% of all transactions it flags as fraud are actually fraudulent (precision). Fig. 2 shows the corresponding confusion matrix, reconstructed from the classification report counts (TP = 88, FN = 10, FP = 1,631, TN = 55,233), which independently reproduces the reported F1-score of 0.0969.

VII. Discussion

A. Interpretation: The Precision Collapse

The results in Table II present an apparent contradiction that is, in fact, a well-known pitfall in imbalanced classification: the proposed model has the *best* ROC-AUC (0.9764) and the *worst* F1-score (0.0969) among all four models. This occurs because:

1) **ROC-AUC measures ranking quality, not calibration.** It evaluates whether fraud cases receive higher predicted probabilities than legitimate cases across *all* thresholds, and remains high even if the single threshold actually used for prediction (0.5) is poorly calibrated.
2) **SMOTE shifts the effective decision surface.** Training on a 50/50 SMOTE-rebalanced set teaches the model that fraud is far more common than it is in reality (0.17%). At inference time, the model applies this rebalanced expectation to the original, highly imbalanced test distribution.
3) **The cost-sensitive weight compounds the effect.** The `scale_pos_weight` penalty was computed from the *original* imbalance ratio ($w_{pos} \approx 578$) and applied *on top of* an already-balanced SMOTE training set, effectively double-penalizing missed fraud cases and pushing the model toward flagging far more transactions as fraudulent than warranted at the default threshold.

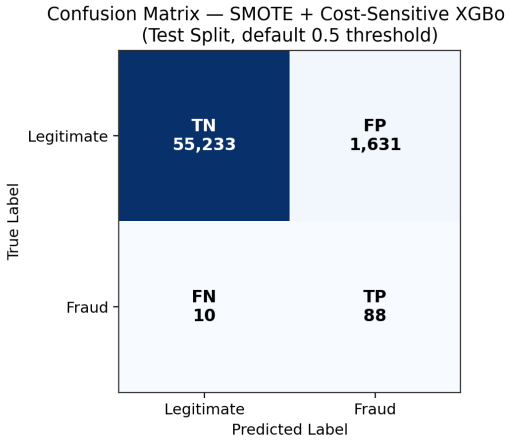

Fig. 2: Confusion matrix for the proposed model, reconstructed from the notebook's classification report at the default 0.5 threshold.

This is precisely the scenario Fawcett [3] warned against: ROC-AUC alone would suggest the proposed model is the best classifier of the four, while the classification report reveals it would flag roughly 1 in every 33 transactions as fraud in production, with only a 1-in-20 chance that any given flagged transaction is genuinely fraudulent.

B. Limitations

1) **No threshold tuning in the evaluation notebook.** All reported test-set metrics use the default 0.5 threshold; no precision-recall curve or optimal threshold search was performed within the notebook's evaluation cells, despite AUPRC/PR-curve analysis being named as an intended metric in the experimental design.
2) **AUPRC not computed.** Although `precision_recall_curve` was imported, no AUPRC value was computed in the final model comparison; only F1 and ROC-AUC are available for direct cross-model comparison.
3) **5-Fold Stratified Cross-Validation not executed for the proposed model.** It is documented as part of the intended validation strategy but is not present in the executed cells; reported numbers reflect a single stratified train/test split.

4) **Deployment/evaluation mismatch.** The Streamlit application (Section VIII) applies an operational 15% probability threshold, which is not the same threshold used to compute the metrics in Table III (0.5). The two are therefore not directly comparable, and the deployed application's real-world precision/recall has not yet been separately measured.
5) **Limited feature exposure in the deployed app.** The Streamlit interface only exposes four of the 28 PCA components (V1–V4) as user-adjustable inputs; V5–V28 are zero-filled by default, which does not reflect a real transaction's full feature vector.

C. Failure Case Analysis

The dominant failure mode is false positives (1,631 of 56,864 legitimate transactions misclassified as fraud) rather than false negatives (only 10 of 98 fraud cases missed). This is the opposite failure profile from the unhandled baselines, which under-predict fraud but do not exhibit a comparable false-positive explosion. This asymmetry is a direct, measurable consequence of combining SMOTE and cost-sensitive weighting without re-calibrating the decision threshold, and is the clearest actionable finding of this study.

VIII. Interactive Deployment Architecture

The trained model (`fraud_model.pkl`) and the amount scaler (`scaler.pkl`, fit only on the `Amount` feature) were embedded in a Streamlit web application. Its actual behavior, as implemented, is as follows:

1) **Inputs.** The user supplies a transaction `Amount`, an elapsed `Time` (seconds), and four PCA components (V1–V4) via sliders in the range $[-10, 10]$.
2) **Scaling.** `Amount` is transformed using the loaded `scaler.pkl`. `Time` is standardized using hardcoded constants (mean $= 94{,}813.86$, std $= 47{,}488.15$) matching the training set's time statistics, rather than a separately serialized time-scaler object.
3) **Feature vector construction.** A 30-dimensional vector is built with indices 0–3 set to the user-provided V1–V4 values, indices 4–27 (V5–V28) left at zero, index 28 set to the scaled amount, and index 29 set to the scaled time.
4) **Inference and thresholding.** The model returns a fraud probability; the transaction is flagged as high risk if the model's binary prediction is 1 *or* the fraud probability is $\geq 15\%$ — an operational threshold chosen to be more conservative than the model's own default 0.5 cutoff, favoring recall.
5) **Output.** The interface displays the fraud probability and a corresponding recommendation (flag for analyst review / step-up authentication, or approve automatically).

Because the zero-filled V5–V28 features and the partial feature exposure do not reflect real transactions, and because the 15% threshold has not been evaluated against the test set using the same classification-report methodology as Table III, the deployed application should currently be regarded as a functional proof-of-concept for the inference pipeline rather than a validated production risk score.

IX. Conclusion & Future Work

This study implemented and evaluated four fraud-detection classifiers — Logistic Regression, Random Forest, standard XGBoost, and a proposed SMOTE-enhanced Cost-Sensitive XGBoost model — on the Kaggle Credit Card Fraud dataset. The proposed model achieved the highest ROC-AUC (0.9764) and the highest fraud recall (89.8%) of all four models, but its F1-score (0.0969) was substantially lower than every baseline due to a precision collapse (5.1%) at the default 0.5 decision threshold. This outcome is not a data or code error but a demonstrable consequence of stacking full SMOTE rebalancing with an XGBoost cost-sensitivity penalty without subsequent threshold calibration, and it concretely illustrates the ROC-AUC limitation on imbalanced data described by Fawcett [3].

Immediate future work should: (1) perform a precision-recall/AUPRC analysis and threshold sweep on the proposed model to identify an operating point that balances recall and precision, rather than relying on the default 0.5 cutoff; (2) re-evaluate the model at the 15% threshold currently used in the Streamlit application, using the same classification-report methodology as Table III, to obtain a validated deployment-time precision/recall estimate; (3) execute the 5-Fold Stratified Cross-Validation described in the experimental design to obtain variance estimates around the reported metrics; (4) expose the full V1–V28 feature set in the deployed application rather than zero-filling 24 of 28 PCA components; and (5) explore alternative or lighter-weight imbalance-handling strategies (e.g., cost-sensitive weighting alone, without SMOTE, or a smaller SMOTE oversampling ratio) to determine whether the precision collapse can be avoided at the source rather than corrected post hoc via thresholding.

Acknowledgment

The author thanks It Simplera Solutions for providing the internship opportunity and dataset access that made this research possible.

References

[1] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
[2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
[3] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
[4] W. Wei, J. Li, L. Cao, A. Ou, and J. Chen, "Effective fraud detection for asymmetric credit card data," *Expert Systems with Applications*, vol. 40, no. 11, pp. 4462–4472, 2013.
[5] F. Carcillo, A. Dal Pozzolo, Y. A. Le Borgne, O. Caelen, Y. Mazzer, and G. Bontempi, "Scarff: a scalable framework for streaming credit card fraud detection with Spark," *Information Fusion*, vol. 41, pp. 182–194, 2018.
[6] Machine Learning Group – ULB, "Credit Card Fraud Detection Dataset," Kaggle, 2018. [Online]. Available: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

[7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
[8] G. Lemaître, F. Nogueira, and C. K. Aridas, “Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning,” *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017.
[9] Streamlit Inc., “Streamlit: The fastest way to build data apps,” 2023. [Online]. Available: https://streamlit.io
[10] T. Chen, T. He, M. Benesty, et al., “xgboost: Extreme Gradient Boosting,” Python package, 2023. [Online]. Available: https://xgboost.readthedocs.io